

SD Basic - Numeric Tests and Formatting

Reducing arrays to a number, testing what is numeric, bit operations, checksums and print formatting.

This page continues SD Basic - Math Functions.

Reducing a dynamic array to a number

These take a dynamic array and give a single value, or a shallower array.

```
sum(array)
summation(array)
maximum(array)
minimum(array)
```

sum()	adds the elements at the lowest mark level present and returns an array one level shallower. Subvalues become values, values become fields
summation()	adds every element at every level and returns one number
maximum()	the largest numeric element
minimum()	the smallest numeric element

WORKED THROUGH

With `a` holding `1`, `2`, `3` as values, and `b` holding a field of `10s20` and `30`, where ^s is a subvalue mark:

EXPRESSION	RESULT
sum(a)	6
sum(b)	30 and 30 as two values — the subvalues collapsed, the value did not
summation(b)	60
maximum(a)	3

EXPRESSION

RESULT

minimum(a)

1

All four silently skip anything that is not a number. With 1, two, 3 in an array, sum() returns 4 and maximum() returns 3. Nothing is reported. **If a non-numeric element means your data is wrong, you have to test for it yourself** — walk the array with num() before summing.

maximum() and minimum() return the **null string**, not zero, when there is no numeric element at all. Test for that with if result = '' then, because a numeric test would read the null string as zero and cannot tell it from a genuine maximum of zero.

While developing, option non.numeric.warning makes SD report a non-numeric value used where a number was expected, which turns a class of silent wrong answers into something you can see. It is off by default. option on its own lists every option and its current state; option all off clears them.

Negative values are handled correctly: maximum() of -5 and -3 is -3.

COMPARING TWO VALUES

```
max(expr1, expr2)
min(expr1, expr2)
```

These take **two values, not an array**, and return the larger or smaller. Unlike the four above they are not restricted to numbers: they use SD's general comparison, so max('abc', 'abd') returns abd. max(3, 7) returns 7.

Testing whether a value is a number

num(expr)

Returns true if *expr* would be accepted as a number, false otherwise.

WHAT IT ACCEPTS

EXPRESSION

RESULT

num('12')

true

num('12.5')

true

EXPRESSION	RESULT	
<code>num('.5')</code>	true	a leading decimal point is fine
<code>num('+3')</code>	true	a leading sign is fine
<code>num('')</code>	true	the null string is numeric
<code>num(' 12')</code>	false	no leading space
<code>num('12 ')</code>	false	no trailing space
<code>num('3-')</code>	false	a trailing sign is not accepted
<code>num('1e3')</code>	false	no exponent notation
<code>num('123456789012345')</code>	true	15 digits before the point
<code>num('1234567890123456')</code>	false	16 is too many

`num('')` is true, and it is the single most common source of a wrong answer from this function. An empty field passes the test and then evaluates as zero, so a validation written as `if num(value) then ...` accepts a blank. **Test for the null string first:**

```
if value # '' and num(value) then ... end
```

The rule is consistent with the rest of SD, where an empty value is zero in a numeric context — but the function reads like a validator and is not one.

`nums()` applies the same test to every element of a dynamic array. See SD Basic - Dynamic Arrays for the whole family of `s` functions.

Bit operations

These work on **32-bit signed integers**. A value outside that range is truncated to fit.

<code>bitand(expr1, expr2)</code>	<code>bitset(expr, bit)</code>
<code>bitor(expr1, expr2)</code>	<code>bitreset(expr, bit)</code>
<code>bitxor(expr1, expr2)</code>	<code>bittest(expr, bit)</code>
<code>bitnot(expr)</code>	<code>shift(expr, distance)</code>

<code>bitand()</code>	bitwise AND. <code>bitand(12, 10)</code> is <code>8</code>
<code>bitor()</code>	bitwise OR. <code>bitor(12, 10)</code> is <code>14</code>
<code>bitxor()</code>	bitwise exclusive OR. <code>bitxor(12, 10)</code> is <code>6</code>
<code>bitnot()</code>	bitwise NOT — every bit inverted. <code>bitnot(0)</code> is <code>-1</code>
<code>bitset()</code>	returns <i>expr</i> with the given bit set. <code>bitset(0, 3)</code> is <code>8</code>
<code>bitreset()</code>	returns <i>expr</i> with the given bit cleared. <code>bitreset(15, 0)</code> is <code>14</code>
<code>bittest()</code>	true if the given bit is set. <code>bittest(8, 3)</code> is true, <code>bittest(8, 2)</code> is false
<code>shift()</code>	shifts the bits. A negative distance shifts left, a positive distance shifts right

Bits are numbered from zero, and zero is the least significant bit. So bit 3 has the value 8, and bit 31 is the sign bit. **There is no range check on the bit number** — a value outside 0 to 31 gives an unpredictable result rather than an error, so validate it yourself if it is calculated.

`shift()` 's direction is the reverse of what the name suggests, and its right shift does not preserve the sign. `shift(1, -4)` is `16` — shifted left. `shift(256, 4)` is `16` — shifted right. And `shift(-1, 1)` is `2147483647`, not `-1`: the value is treated as unsigned, so a right shift brings in zeros rather than copying the sign bit. **Do not use `shift()` to divide a signed number by a power of two** — use `div()`.

Checksums

`checksum(expr)`

Returns a 32-bit integer derived from the bytes of *expr*. It is cheap and it changes when the data changes — `checksum('hello')` is `1199` and `checksum('hellp')` is `1200` — which makes it useful for spotting that a record has been altered since you last read it.

It is not a cryptographic hash and must not be used as one. It is a rotate-and-exclusive-or over the bytes, and it is straightforward to construct a different string with the same result. It will not detect deliberate tampering, and it must never be used to store or compare a password. `checksum('')` is `0`, so an empty value and a failed read are indistinguishable by checksum alone.

For real cryptography this port provides `sdencrypt()` and `sddecrypt()`. The older `encrypt()` and `decrypt()` functions **have been removed** and programs using them will not compile.

Controlling how numbers print

precision *n*

Sets how many decimal places are used when a floating-point number is converted to a string — by `print`, by concatenation, by writing to a file. It does not change the accuracy of the arithmetic itself, only what you see and what gets stored.

The default is 4. *n* may be 0 to 14; a value outside that range is pulled to the nearest end rather than refused.

<code>1 / 3</code> at the default	<code>0.3333</code>
<code>1 / 3</code> after <code>precision 8</code>	<code>0.33333333</code>
<code>1 / 3</code> after <code>precision 0</code>	<code>0</code>
<code>1 / 3</code> after <code>precision 20</code>	<code>0.33333333333333</code> — clamped to 14, not refused

Precision is per program and it does not travel with a call. Every program starts at 4, including one you `call`, and setting it in a caller has no effect inside the subroutine. It is restored to the caller's value when the subroutine returns. **A subroutine that formats numbers must set its own `precision`** — a common cause of a report where the totals show four decimal places and the line items show something else.

Where you need a specific layout rather than a number of decimals, use `fmt()` and the output conversion codes instead; they are covered in SD Basic - Data Conversion.

COMPARING FLOATING-POINT NUMBERS

Two floating-point values are treated as equal if they differ by less than the `FLTDIFF` configuration parameter, which defaults to a very small number (`0.0000000000291`).

This is why `0.1 + 0.2 = 0.3` is true in SD BASIC, where the same test is false in C, Java, Python and JavaScript. It is a deliberate convenience and not an accident of this port. `config('FLTDIFF')` reports the value in force.

It also means two values that differ in the fourteenth decimal place compare as equal, so a comparison is not a safe way to detect a tiny drift. If you need to know that two numbers really are bit-identical, compare their string forms after `precision 14`.

Applying a function to every element

Several of the functions above have a partner ending in `s` that applies the same operation to every element of a dynamic array instead of to a single value: `abss()`, `negs()`, `mods()`, `nums()`.

```
abss(-1 : @vm : 2 : @vm : -3)    ;* gives 1, 2, 3 as three values
```

The whole family — including the string and comparison members — is described in SD Basic - Dynamic Arrays.

What is not here

These existed in OpenQM and in earlier SD releases and **are not in SD Core for Windows**. A program using one will not compile.

`encrypt()` · `decrypt()`

replaced by `sdencrypt()` and `sddecrypt()`

Nothing else in the mathematical and logical group has been removed: every function the OpenQM 2.6.6 documentation lists under that heading is present in this port, and this page covers the numeric ones. The comparison and logical members of the group — `ands()`, `ors()`, `nots()`, `eqs()`, `nes()`, `gts()`, `ges()`, `lts()`, `les()`, `ifs()`, `not()`, `reuse()` — are documented in SD Basic - Dynamic Arrays, where their multivalued behaviour is the point.

See also

SD Basic - Dynamic Arrays · SD Basic - Data Conversion · SD Basic - String Functions · the `config` verb, for `INTPREC` and `FLTDIFF`.
